PROYECTO DE AULA / OCKLY

Desarrollo de una aplicación web con MVC y Spring Boot

Documento técnico y manual del sistema

Datos del proyecto	Descripción
Aplicación	Ockly – Gestión de categorías y productos de papelería
Asignatura	Arquitectura de Software
Institución	Institución Universitaria Pascual Bravo
Arquitectura	Modelo–Vista–Controlador (MVC)
Tecnologías	Java 17 · Spring Boot 3.5.16 · Thymeleaf · JPA · MySQL
Año	2026

Integrantes

Sebastián Pineda · Melissa Betancur Osorio · Miguel Ángel Noreña Cano

Repositorio

https://github.com/drakcore12/prueba-github/tree/ockly-mvc-springboot

El código fue publicado en una rama independiente del repositorio de pruebas; la rama principal no fue modificada.

1. Descripción del proyecto

Ockly es una aplicación web académica para organizar el catálogo de una papelería. Permite administrar categorías y productos en una base de datos relacional y operar desde el navegador mediante formularios y tablas.

1.1 Problema y objetivo

El registro manual o disperso de artículos dificulta localizar productos, conocer existencias y mantener una clasificación consistente. El objetivo es centralizar esta información y demostrar MVC, CRUD, validaciones y persistencia en MySQL.

1.2 Funcionalidades implementadas en el código

- CRUD completo de categorías: crear, listar, consultar, editar y eliminar.
- CRUD completo de productos: crear, listar, consultar, editar y eliminar.
- Relación de base de datos: cada producto pertenece a una categoría y una categoría contiene varios productos.
- Validaciones: nombres obligatorios, precio positivo, stock no negativo y selección de categoría.
- Regla de integridad: no eliminar una categoría mientras contenga productos.
- Interfaz responsive con tablas, formularios, mensajes y panel de inicio.

2. Arquitectura y organización del proyecto

Se utiliza Modelo–Vista–Controlador con capas auxiliares de servicio y repositorio. Controller recibe HTTP y prepara la Vista. Service concentra operaciones y reglas; Repository accede a MySQL mediante JPA; Model define las entidades persistentes.

Componente	Paquete o ubicación	Responsabilidad
Modelo	model/	Entidades Categoria y Producto; asociación JPA uno a muchos.
Vista	templates/ + static/css/	HTML Thymeleaf y estilos; formularios y tablas.
Controlador	controller/	Rutas y respuesta MVC; renderizado y navegación.
Servicios	service/	Reglas CRUD y restricciones de eliminación.
Persistencia	repository/	Consultas y escritura mediante Spring Data JPA.
Formulario	dto/	Captura y validación de los datos de Producto.

Secuencia principal: Navegador → Controller → Service → Repository → MySQL. El Controller entrega un Model a Thymeleaf, que produce el HTML de respuesta.

2.1 Modelo relacional

CATEGORIAS (id PK, nombre, descripcion) 1 —— N PRODUCTOS (id PK, nombre, descripcion, precio, stock, categoria_id FK).

La relación existe tanto en Java mediante @OneToMany y @ManyToOne como en SQL mediante la clave foránea fk_productos_categorias (ON DELETE RESTRICT).

3. Manual de instalación y configuración

3.1 Requisitos

Herramienta	Requisito
JDK	Java 17 o 21
Maven	3.6.3 o superior
Framework	Spring Boot 3.5.16 (pom.xml)
Motor relacional	MySQL 8 / MariaDB de XAMPP reciente
IDE	VS Code, IntelliJ o Eclipse
Navegador	Chrome, Firefox o Edge

3.2 Crear base de datos

1. Iniciar MySQL desde el panel de XAMPP. Apache es opcional para usar phpMyAdmin.
2. Abrir http://localhost/phpmyadmin y entrar a la pestaña SQL.
3. Ejecutar completo el archivo database/schema.sql del repositorio. El archivo crea la base ockly_db y las tablas categorias y productos.
4. Comprobar que ambas tablas existen y que productos.categoria_id tiene clave foránea.

3.3 Parámetros de conexión

Parámetro	Valor
Motor / base de datos	MySQL / ockly_db
Host y puerto SQL	localhost:3306
Usuario por defecto	root
Contraseña	Variable DB_PASSWORD (por defecto vacía)
Puerto web	8080
Archivo de configuración	src/main/resources/application.properties

Si existe contraseña de MySQL, defina DB_PASSWORD en su terminal. No se deben almacenar contraseñas personales en el repositorio.

3.4 Compilar y ejecutar

```
java -version
mvn -version
mvn clean spring-boot:run
Abrir http://localhost:8080/
```

La conexión por defecto se define como jdbc:mysql://localhost:3306/ockly_db. Si hay un error de acceso, verifique credenciales; si falta el esquema, ejecute nuevamente database/schema.sql. El proyecto no se ejecuta con MySQL apagado.

3.5 Pruebas

Ejecutar mvn test. El archivo OcklyMvcIntegrationTest contiene pruebas HTTP del CRUD, la asociación entre entidades y las validaciones. H2 solo se utiliza en el perfil aislado de pruebas; el sistema normal se conecta a MySQL.

4. Rutas y guía de uso

Funcionalidad	Dirección (localhost:8080)
Inicio	/
Listar categorías	/categorias
Crear categoría	/categorias/nuevo
Ver categoría	/categorias/{id}
Editar categoría	/categorias/{id}/editar
Listar productos	/productos
Crear producto	/productos/nuevo
Ver producto	/productos/{id}
Editar producto	/productos/{id}/editar

El identificador {id} es el valor real registrado en MySQL. Para eliminar se utilizan formularios POST, no enlaces GET.

4.1 Administración de categorías

5. En Categorías, hacer clic en Nueva categoría.
6. Escribir nombre y descripción opcional; pulsar Guardar categoría.
7. En el listado, seleccionar Consultar para ver detalle y productos asociados.
8. Usar Editar para modificar y Eliminar para borrar categorías sin productos.

4.2 Administración de productos

9. Crear primero una categoría para poder asociar el producto.
10. Entrar en Productos → Nuevo producto.
11. Registrar nombre, precio mayor que cero, stock no negativo y seleccionar una categoría.
12. Guardar y comprobar su aparición en la tabla; usar Consultar, Editar o Eliminar.
13. Verificar la asociación entrando a Detalle de categoría y observando los productos.

5. Evidencias de funcionamiento pendientes

Las capturas exigidas por el docente deben tomarse desde la aplicación realmente ejecutada en un navegador con MySQL configurado. Para evitar presentar evidencia ficticia, este documento registra expresamente las capturas que deben incorporarse antes de entregar la versión final.

- Captura 1. Pantalla de inicio, con recuento de categorías y productos.

- Captura 2. Formulario de creación y tabla de categorías.
- Captura 3. Formulario de productos con selector de categoría y tabla CRUD.
- Captura 4. Detalle de categoría y productos vinculados.
- Captura 5. phpMyAdmin, tablas y relación de clave foránea.
- Captura 6. Resultado de mvn test o compilación.

5.1 Registro de evidencias a incorporar

Evidencia	Cómo obtenerla
Inicio	Abrir / con categorías y productos ya registrados.
CRUD categorías	Mostrar /categorias, crear una categoría, editarla.
CRUD productos	Mostrar /productos, crear, editar y borrar un producto.
Relación	Abrir /categorias/{id} y ver productos vinculados.
Persistencia SQL	Consultar ambas tablas en phpMyAdmin; comprobar categoria_id.
Pruebas	Ejecutar mvn test y conservar salida final de la terminal.

Estado de verificación: el código fuente, la estructura y la documentación están preparados y publicados. En este entorno no se pudo ejecutar Maven/MySQL localmente ni obtener capturas auténticas de la aplicación. El flujo CI de GitHub no llegó a asignar un runner; no hay resultado de prueba confirmado. Las evidencias operativas deben completarse tras el arranque en el equipo del grupo.

6. Entrega

Entregar: (1) el enlace a la rama GitHub con código; (2) el proyecto comprimido; (3) este documento en Word con capturas reales agregadas; (4) la base SQL incluida en el repositorio.